

Week 6 – Apache Spark (PySpark)

Assignment Report

Objective

The objective of this week's assignment is to understand Apache Spark's distributed architecture and to perform efficient, large-scale data processing using DataFrame transformations, filtering, schema handling, and optimized file formats. This includes reading and writing CSV and Parquet files, applying column-level transformations (rename, cast, derive), filtering with single and compound conditions, and understanding the performance implications of shuffle operations and predicate pushdown.

Introduction to Apache Spark

Apache Spark is an open-source, distributed data processing engine designed for large-scale data analytics. Unlike traditional single-machine tools (like pandas), Spark splits data and computation across many machines (a cluster), allowing it to process datasets far larger than what fits in the memory of a single computer.

Spark exposes several APIs — RDDs (low-level), DataFrames, and Datasets (Scala/Java) — but for most day-to-day data engineering work, the DataFrame API (used throughout this assignment via PySpark) is preferred because it is easier to use and benefits from Spark's Catalyst query optimizer and Tungsten execution engine.

Spark Architecture

A Spark application follows a master–worker architecture made up of three main components: the Driver, the Cluster Manager, and the Executors.

Driver

The Driver is the process that runs the `main()` function of the Spark application. It creates the `SparkSession/SparkContext`, converts the user's code into a logical plan and then a physical plan (a DAG of stages and tasks), and coordinates the executors. The Driver also collects results back from executors when actions like `.collect()` are called. If the Driver process dies, the whole application dies with it — which is why in production it is common to run the Driver itself under supervision (e.g. cluster mode with a cluster manager that can restart it).

Cluster Manager

The Cluster Manager is responsible for allocating resources (CPU, memory) across applications running on the cluster. Spark supports several: Standalone (Spark's own simple manager), YARN (common in Hadoop ecosystems), Kubernetes, and Mesos. The Driver asks the Cluster Manager for executors; the Cluster Manager decides which physical machines/containers those executors run on.

Executors

Executors are worker processes launched on cluster nodes. Each executor runs the actual tasks assigned to it, stores data in memory/disk for caching, and reports status and results back to the Driver. Executors run for the lifetime of the application (unless dynamic allocation is enabled) and can run multiple tasks in parallel using multiple cores.

Execution Modes

Spark applications can run in Client mode or Cluster mode, and this matters for where the Driver lives relative to the cluster (covered in detail in Q13).

Lazy Evaluation

Spark transformations (filter, select, map, withColumn, etc.) are not executed immediately. Instead, Spark builds up a logical execution plan describing what should happen. Actual computation only happens when an action (show, count, collect, write, etc.) is called. This is Spark's lazy evaluation model.

DAG (Lineage Graph)

As transformations are chained, Spark records them as a Directed Acyclic Graph (DAG) of stages, where each node represents an RDD/DataFrame transformation and edges represent dependencies. This DAG — also called the lineage graph — is the record of how each piece of data was derived, and Spark's Catalyst optimizer can rearrange and optimize it (e.g. pushing filters earlier) before executing.

Transformations

Transformations are lazy operations that produce a new DataFrame/RDD from an existing one, without immediately computing anything (e.g. filter, select, withColumn, groupBy, join).

Actions

Actions trigger the actual execution of the DAG built up by prior transformations, and return a result to the Driver or write data out (e.g. show, count, collect, write, take).

CSV vs Parquet

CSV is a row-based, plain-text format: every value is read as a string and the whole row must be parsed to reach any single column. Parquet is a columnar, binary, typed, and compressed format: it stores data column by column with per-column statistics, so Spark can skip entire columns or row-groups that a query doesn't need. This generally makes Parquet noticeably faster to read and smaller on disk than CSV, especially for wide tables where only a few columns are queried at a time.

Predicate Pushdown

Predicate pushdown is an optimization where filter conditions in a query are pushed down into the data source's own file scan, so that unnecessary data is skipped before it is even loaded into Spark's execution engine. Parquet supports this well because it stores min/max statistics per row-group, letting Spark's reader skip whole row-groups that cannot match the filter. CSV does not support predicate pushdown because rows must be parsed sequentially before their column values are even known.

Shuffle

A shuffle is the redistribution of data across partitions/executors over the network, required by operations like groupBy, join, distinct, and orderBy when the relevant keys aren't already co-located. Shuffles are expensive (network I/O, disk I/O, serialization) and are usually the biggest performance cost in a Spark job, so minimizing unnecessary shuffles is a key optimization technique.

Schema Handling

A schema describes the structure of a DataFrame: column names, data types, and nullability. Spark can infer a schema automatically (`inferSchema=True` for CSV, or automatically for Parquet, since Parquet stores its schema in the file itself), or a schema can be defined explicitly using `StructType/StructField`. Explicit schemas are recommended for large or production datasets because `inferSchema` requires an extra pass over the data and can guess types incorrectly.

Data Pipeline

A typical Spark data pipeline follows a read -> transform -> filter -> write pattern: data is read from a source (CSV/Parquet/database), cleaned and transformed (rename, cast, derive columns, handle nulls), filtered down to what's relevant, and finally written out to a sink in an optimized format such as Parquet.

Week 6 Questions — Detailed Answers

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

1. Concept Explanation

A Spark application is coordinated by the Driver, which runs the user's main program, builds the DAG of transformations, and schedules work.

The Cluster Manager (Standalone, YARN, Kubernetes, or Mesos) allocates cluster resources and launches Executor processes on worker nodes on the Driver's behalf.

Executors run the actual tasks (units of work operating on a data partition), cache data in memory/disk, and send results/status back to the Driver.

PySpark Code

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName("DriverClusterExecutorDemo")
        .master("local[*]")
        .getOrCreate()
)

# The SparkContext exposes cluster-level info that ultimately comes
# from the Cluster Manager's resource allocation to this application
sc = spark.sparkContext
print("Default parallelism (executor cores available):", sc.defaultParallelism)
print("Application ID:", sc.applicationId)
```

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

1. Concept Explanation

With lazy evaluation, calling a transformation (filter, select, withColumn, etc.) does not execute anything immediately — Spark just records it as a step in a logical plan.

Only when an action (show, count, collect, write) is called does Spark's Catalyst optimizer analyze the entire chain of transformations together and compile an optimized physical plan.

PySpark Code

```
df = spark.read.option("header", True).option("inferSchema",
True).csv("data/source.csv")

# Nothing has executed yet -- these are all lazy transformations
step1 = df.filter(df.category == "Electronics")
step2 = step1.select("product_id", "price")
step3 = step2.withColumn("price_with_tax", step2.price * 1.18)

# Execution only happens now, when an action is called
step3.show(5)
```

Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

1. Concept Explanation

Spark's DataFrameReader (spark.read) provides a fluent .option() API to control how a file is parsed, including header handling and schema inference.

PySpark Code

```
df = (
    spark.read
        .option("header", True)
        .option("inferSchema", True)
        .csv("data/source.csv")
)

df.printSchema()
```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

1. Concept Explanation

CSV stores data row by row as plain text: to read even a single column, Spark must read and parse every field in every row.

Parquet stores data column by column in a compressed, binary, typed format, with metadata (including min/max statistics) recorded per column per row-group.

PySpark Code

```
# Write the same DataFrame in both formats to compare
df.write.mode("overwrite").option("header",
True).csv("output/csv_output/compare_csv")
df.write.mode("overwrite").parquet("output/parquet_output/compare_parquet")

# Reading back only two columns
csv_cols = spark.read.option("header", True).option("inferSchema", True) \
    .csv("output/csv_output/compare_csv").select("category", "price")

parquet_cols = spark.read.parquet("output/parquet_output/compare_parquet") \
    .select("category", "price")
```

Q5. Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

1. Concept Explanation

This combines a transformation to filter rows (filter/where) with a transformation to project specific columns (select).

3. PySpark Code

```
electronics = df.filter(df.category == "Electronics").select("product_id", "price")
electronics.show()

# Equivalent alternatives:
# df.where(df["category"] == "Electronics").select("product_id", "price")
# df.select("product_id", "price").filter(F.col("category") == "Electronics")
```

Q6. Write the code to rename the column old_name to new_name and cast the price column from String to Double.

1. Concept Explanation

withColumnRenamed(existing, new) renames a column; .withColumn(name, col.cast(type)) overwrites a column with a type-cast version of itself.

PySpark Code

```
from pyspark.sql.types import DoubleType
from pyspark.sql import functions as F

df_clean = (
    df.withColumnRenamed("old_name", "new_name")
      .withColumn("price", F.col("price").cast(DoubleType()))
)

df_clean.select("new_name", "price").printSchema()
```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

1. Concept Explanation

Every RDD/DataFrame in Spark remembers how it was derived from its parent(s) — this chain of transformations is the lineage graph, structured as a DAG (Directed Acyclic Graph).

Because Spark keeps this recipe rather than only the resulting data, if a partition of data is lost (e.g. an executor/worker node crashes), Spark can recompute just that lost partition by re-running the relevant portion of the lineage on another worker.

PySpark Code

```
df = spark.read.option("header", True).option("inferSchema",
True).csv("data/source.csv")

filtered = df.filter(F.col("status") == "Completed")
transformed = filtered.withColumn("final_price", F.col("base_price") * 1.18)

# Inspect the recorded lineage / physical plan Spark would use to
# recompute 'transformed' from scratch if a partition were lost
transformed.explain(True)
```

Q8. Write a query to filter df_orders where status = 'Completed' AND amount > 1000.

1. Concept Explanation

Combines two conditions with a logical AND (&) inside a single .filter()/where() call using Spark Column expressions.

3. PySpark Code

```
df_orders = df # using the same sample dataset as df_orders for this exercise

high_value_completed = df_orders.filter(
    (F.col("status") == "Completed") & (F.col("amount") > 1000)
)

high_value_completed.select("product_id", "status", "amount").show()
```


Q9. Explain Predicate Pushdown in Parquet.

1. Concept Explanation

Predicate pushdown means the filter conditions (predicates) written in a Spark query are passed down into the file format's own reader, so filtering happens as data is being read from disk rather than after it's all loaded into Spark.

Parquet supports this well because each file stores column-level statistics (min, max, and sometimes null counts) per row-group, letting the reader decide whether a whole row-group could possibly contain matching rows before decoding it.

PySpark Code

```
df_parquet = spark.read.parquet("output/parquet_output/source.parquet")

expensive_electronics = df_parquet.filter(
    (F.col("category") == "Electronics") & (F.col("price") > 500)
)

expensive_electronics.explain(True)
```

Q10. Add a new column final_price = base_price × 1.18.

1. Concept Explanation

withColumn(name, expression) adds a new column (or overwrites an existing one with the same name) computed from an expression over existing columns.

PySpark Code

```
df_with_final_price = df.withColumn(
    "final_price", F.round(F.col("base_price") * 1.18, 2)
)

df_with_final_price.select("product_id", "base_price", "final_price").show(5)
```

Q11. Difference between Transformations and Actions with two examples each.

1. Concept Explanation

Transformations are lazy: calling one just adds a step to Spark's logical plan and returns a new DataFrame/RDD immediately, without processing any data. Examples: .filter(), .select(), .withColumn(), .groupBy(), .join(), .distinct().

Actions are eager: calling one triggers Spark to actually execute the accumulated DAG of transformations and either returns a result to the driver or writes data to a sink. Examples: .show(), .count(), .collect(), .write.csv(/parquet), .take(n).

PySpark Code

```
# Two transformations (lazy -- nothing executes yet)
filtered = df.filter(F.col("category") == "Electronics") # transformation
grouped = df.groupBy("category").agg(F.avg("amount").alias("avg_amount")) # transformation

# Two actions (trigger execution)
row_count = df.count() # action
top_rows = filtered.take(3) # action
```

Q12. Read a Parquet file from "path/to/input", remove rows where user_id is null, and save the result as CSV in "path/to/output".

1. Concept Explanation

This is a small end-to-end pipeline: read (Parquet) -> transform/filter (drop null user_id rows) -> write (CSV) — the read -> transform -> filter -> write pattern referenced in the assignment steps.

PySpark Code

```
df_input = spark.read.parquet("path/to/input")

df_clean = df_input.na.drop(subset=["user_id"])

df_clean.write.mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13. Difference between Client Mode and Cluster Mode.

1. Concept Explanation

In Client Mode, the Driver runs on the machine that submitted the application (e.g. your laptop, an edge node, or a notebook server) — outside the cluster — while Executors run on the cluster.

In Cluster Mode, the Driver itself is also launched inside the cluster (on one of the worker nodes), managed by the Cluster Manager, alongside the Executors.

PySpark Code

```
# Illustrative spark-submit commands (not run inside this notebook,
# since this notebook itself runs in local[*] mode)

# Client mode: driver runs on the machine issuing the command
# spark-submit --master yarn --deploy-mode client my_job.py

# Cluster mode: driver runs inside the cluster
# spark-submit --master yarn --deploy-mode cluster my_job.py
```

Q14. Filter rows where region = 'North' OR priority = 'High'.

1. Concept Explanation

Combines two conditions with a logical OR (|) inside a single filter/where call, keeping a row if either condition is true.

PySpark Code

```
north_or_high_priority = df.filter(
    (F.col("region") == "North") | (F.col("priority") == "High")
)

north_or_high_priority.select("product_id", "region", "priority").show(12)
```

Q15. Why is .show(5) safer than .collect() for very large datasets?

1. Concept Explanation

.show(n) computes and prints only the first n rows (default 20) and never pulls the full dataset to the Driver's memory.

.collect() materializes the ENTIRE DataFrame as a list of Row objects in the Driver's memory — for a dataset with billions of rows, this can easily exceed the Driver's available memory and crash the application with an OutOfMemoryError.

PySpark Code

```
# Safe on datasets of any size -- only computes/prints 5 rows
df.show(5)

# Risky on large datasets -- pulls every row to the Driver's memory
# all_rows = df.collect() # avoid on large data!

# Safer alternative when you truly need data back at the driver:
# aggregate first, THEN collect the small result
summary = df.groupBy("category").agg(F.count("*").alias("cnt")).collect()
```

Conclusion

This assignment built a working understanding of Apache Spark's distributed architecture (Driver, Cluster Manager, Executors) and its execution model, centered on lazy evaluation and the DAG/lineage graph that gives Spark both query optimization opportunities and fault tolerance. Applying these concepts practically — reading and writing CSV and Parquet, handling schemas, renaming and casting columns, adding derived columns, filtering with AND/OR logic, and distinguishing transformations from actions — reinforced why Parquet, predicate pushdown, and careful use of actions like .collect() matter for performance at scale. The accompanying notebook (Week6_Spark_Assignment.ipynb) contains the runnable version of all code shown here, executed against a small sample dataset (data/source.csv) so that every printed output in this report matches what running the notebook produces.